```python
import pandas as pd
```

```python
df=pd.read_excel('skintextdata.xlsx')
```

```python
df
```

	Oil_Level	Water_Intake	Sleep_Hours	Stree_level	Acne_count	lab
0	9	2	4	9.0	17	apollo
1	9	3	5	8.0	16	medplus
2	8	3	5	8.0	14	srl
3	8	4	6	7.0	13	Viyaya
4	7	4	6	7.0	11	apollo
5	7	5	7	6.0	10	medplus
6	6	5	7	6.0	9	srl
7	6	6	7	5.0	8	medplus
8	5	6	8	5.0	7	apollo
9	5	7	8	NaN	6	srl
10	4	7	8	4.0	5	Viyaya
11	4	8	9	3.0	4	apollo
12	3	8	9	3.0	3	srl
13	3	9	9	2.0	2	medplus
14	2	9	10	2.0	1	apollo
15	2	10	10	1.0	1	srl
16	1	10	10	1.0	0	apollo

Next steps: Generate code with df New interactive sheet

[10]
✓ 0s

```
dummies=pd.get_dummies(df.lab).astype(int)
```

[11]
✓ 0s

```
dummies
```

	Viyaya	apollo	medplus	srl
0	0	1	0	0
1	0	0	1	0
2	0	0	0	1
3	1	0	0	0
4	0	1	0	0
5	0	0	1	0
6	0	0	0	1
7	0	0	1	0
8	0	1	0	0
9	0	0	0	1
10	1	0	0	0
11	0	1	0	0
12	0	0	0	1
13	0	0	1	0
14	0	1	0	0
15	0	0	0	1
16	0	1	0	0

Next steps: Generate code with dummies New interactive sheet

[12]
✓ 0s

```
merged=pd.concat([df,dummies],axis='columns')
merged
```

[13]

```
merged["Stree_level"]=df["Stree_level"].fillna(4)
```

[14]

```
final = merged.drop(['lab'], axis='columns')
final
```

	Oil_Level	Water_Intake	Sleep_Hours	Stree_level	Acne_count	Viyaya	apollo	medplus	srl
0	9	2	4	9.0	17	0	1	0	0
1	9	3	5	8.0	16	0	0	1	0
2	8	3	5	8.0	14	0	0	0	1
3	8	4	6	7.0	13	1	0	0	0
4	7	4	6	7.0	11	0	1	0	0
5	7	5	7	6.0	10	0	0	1	0
6	6	5	7	6.0	9	0	0	0	1
7	6	6	7	5.0	8	0	0	1	0
8	5	6	8	5.0	7	0	1	0	0
9	5	7	8	4.0	6	0	0	0	1
10	4	7	8	4.0	5	1	0	0	0
11	4	8	9	3.0	4	0	1	0	0
12	3	8	9	3.0	3	0	0	0	1
13	3	9	9	2.0	2	0	0	1	0
14	2	9	10	2.0	1	0	1	0	0
15	2	10	10	1.0	1	0	0	0	1
16	1	10	10	1.0	0	0	1	0	0

Next steps: Generate code with final New interactive sheet

[16]

```
x=final.drop('Acne_count',axis='columns')
```

[16]
```
x=final.drop('Acne_count',axis='columns')
```

[17]
```
x
```

	Oil_Level	Water_Intake	Sleep_Hours	Stree_level	Viyaya	apollo	medplus	srl
0	9	2	4	9.0	0	1	0	0
1	9	3	5	8.0	0	0	1	0
2	8	3	5	8.0	0	0	0	1
3	8	4	6	7.0	1	0	0	0
4	7	4	6	7.0	0	1	0	0
5	7	5	7	6.0	0	0	1	0
6	6	5	7	6.0	0	0	0	1
7	6	6	7	5.0	0	0	1	0
8	5	6	8	5.0	0	1	0	0
9	5	7	8	4.0	0	0	0	1
10	4	7	8	4.0	1	0	0	0
11	4	8	9	3.0	0	1	0	0
12	3	8	9	3.0	0	0	0	1
13	3	9	9	2.0	0	0	1	0
14	2	9	10	2.0	0	1	0	0
15	2	10	10	1.0	0	0	0	1
16	1	10	10	1.0	0	1	0	0

Next steps: Generate code with x   New interactive sheet

[18]
```
y=final["Acne_count"]
y
```

[19]

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

[20]

```
model.fit(x,y)
```

```
LinearRegression
LinearRegression()
```

[21]

```
model.predict(x)
```

```
array([16.76137484, 14.88942105, 14.0429638 , 12.70587407, 11.83842795,
        9.96647415,  9.1200169 ,  8.67784195,  6.91548105,  6.57008029,
        5.29412593,  4.36554444,  2.90843781,  2.46626285,  0.70390196,
        0.3585012 , -0.58473024])
```

[22]

```
model.score(x,y)
```

```
0.9910818354343913
```

[23]

```
model.predict([[0,6,5,0,0,1,0,0]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lin
  warnings.warn(
array([4.21200169])
```

[24]

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

[25]

```
dfle=df
dfle.lab=le.fit_transform(dfle.lab)
```

[26]

```
dfle
```

	Oil_Level	Water_Intake	Sleep_Hours	Stree_level	Acne_count	lab
0	9	2	4	9.0	17	1

```
dfle["Stree_level"]=dfle["Stree_level"].fillna(4)
```

```
dfle
```

	Oil_Level	Water_Intake	Sleep_Hours	Stree_level	Acne_count	lab
0	9	2	4	9.0	17	1
1	9	3	5	8.0	16	2
2	8	3	5	8.0	14	3
3	8	4	6	7.0	13	0
4	7	4	6	7.0	11	1
5	7	5	7	6.0	10	2
6	6	5	7	6.0	9	3
7	6	6	7	5.0	8	2
8	5	6	8	5.0	7	1
9	5	7	8	4.0	6	3
10	4	7	8	4.0	5	0
11	4	8	9	3.0	4	1
12	3	8	9	3.0	3	3
13	3	9	9	2.0	2	2
14	2	9	10	2.0	1	1
15	2	10	10	1.0	1	3
16	1	10	10	1.0	0	1

Next steps: Generate code with df New interactive sheet

```
a=dfle[['Oil_Level','Water_Intake','Sleep_Hours','Stree_level','lab']].values
```

```
array([17, 16, 14, 13, 11, 10,  9,  8,  7,  6,  5,  4,  3,  2,  1,  1,  0])
```

[33] 0s

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer([('lab', OneHotEncoder(),[4])], remainder = 'passthrough')
```

[34] 0s

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

# Corrected ColumnTransformer definition
# Apply OneHotEncoder to the 'lab' column (index 4 in the 'a' array)
# and pass through the other columns
ct = ColumnTransformer([('lab_onehotencoder', OneHotEncoder(), [4])], remainder='passthrough')

X = ct.fit_transform(a)
X
```

```
array([[ 0.,  1.,  0.,  0.,  9.,  2.,  4.,  9.],
       [ 0.,  0.,  1.,  0.,  9.,  3.,  5.,  8.],
       [ 0.,  0.,  0.,  1.,  8.,  3.,  5.,  8.],
       [ 1.,  0.,  0.,  0.,  8.,  4.,  6.,  7.],
       [ 0.,  1.,  0.,  0.,  7.,  4.,  6.,  7.],
       [ 0.,  0.,  1.,  0.,  7.,  5.,  7.,  6.],
       [ 0.,  0.,  0.,  1.,  6.,  5.,  7.,  6.],
       [ 0.,  0.,  1.,  0.,  6.,  6.,  7.,  5.],
       [ 0.,  1.,  0.,  0.,  5.,  6.,  8.,  5.],
       [ 0.,  0.,  0.,  1.,  5.,  7.,  8.,  4.],
       [ 1.,  0.,  0.,  0.,  4.,  7.,  8.,  4.],
       [ 0.,  1.,  0.,  0.,  4.,  8.,  9.,  3.],
       [ 0.,  0.,  0.,  1.,  3.,  8.,  9.,  3.],
       [ 0.,  0.,  1.,  0.,  3.,  9.,  9.,  2.],
       [ 0.,  1.,  0.,  0.,  2.,  9., 10.,  2.],
       [ 0.,  0.,  0.,  1.,  2., 10., 10.,  1.],
       [ 0.,  1.,  0.,  0.,  1., 10., 10.,  1.]])
```

[ ]

```python
model.fit(X,b)
```

n all

```
import pickle
```

[48]

```
with open('model_pickle','wb') as file:
    pickle.dump(model,file)
```

[50]

```
with open('model_pickle','rb') as file:
    mp = pickle.load(file)
```

[51]

```
mp.coef_
```

```
array([ 1.20016904, -0.04423158, -1.17284125,  0.04423158, -0.03264544,
        0.30007748, -0.31057191,  0.04313988])
```

[52]

```
mp.intercept_
```

```
np.float64(10.041519932384839)
```

[53]

```
mp.predict([[0,0,1,0,0,8,10,5]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
array([8.37927877])
```

Double-click (or enter) to edit

[56]

```
import joblib
```

[57]

```
joblib.dump(model, 'model_joblib')
```

```
['model_joblib']
```

[58]

```
mj = joblib.load('model_joblib')
```

[59]

```
mj.coef_
```

```
array([ 1.20016904, -0.04423158, -1.17284125,  0.04423158, -0.03264544,
```

[52]

```
mp.intercept_
```

```
np.float64(10.041519932384839)
```

[53]

```
mp.predict([[0,0,1,0,0,8,10,5]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
array([8.37927877])
```

Double-click (or enter) to edit

[56]

```
import joblib
```

[57]

```
joblib.dump(model, 'model_joblib')
```

```
['model_joblib']
```

[58]

```
mj = joblib.load('model_joblib')
```

[59]

```
mj.coef_
```

```
array([ 1.20016904, -0.04423158, -1.17284125,  0.04423158, -0.03264544,
        0.30007748, -0.31057191,  0.04313988])
```

[60]

```
mj.intercept_
```

```
np.float64(10.041519932384839)
```

[62]

```
mj.predict([[0,0,1,0,0,8,10,5]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
array([8.37927877])
```

ilable